

Cloud-Based Chaos Engineering & Infrastructure Monitoring

Project Documentation

Prepared by: Dhanush V

Role: MCA Graduate | Cloud & DevOps Engineer (Aspiring)

GitHub Repository: [GitHub](#)

Executive Summary

Cloud-Based Chaos Engineering & Infrastructure Monitoring is a practical DevOps project designed to demonstrate cloud infrastructure provisioning, automated deployments, monitoring, and resilience testing on AWS.

The project uses Terraform to provision infrastructure, Docker and Docker Compose to deploy applications, CloudWatch to monitor system performance, and GitHub Actions to automate container image builds. Chaos engineering techniques are applied to simulate infrastructure failures such as CPU spikes, memory exhaustion, disk utilization, and inode exhaustion, enabling analysis of system behavior under stress.

The project showcases industry-relevant skills in Infrastructure as Code (IaC), containerization, cloud monitoring, automation, and troubleshooting.

Project Objectives

The primary objectives of this project are:

- Provision cloud infrastructure using Terraform.
- Deploy containerized applications with Docker and Docker Compose.
- Implement automated deployment workflows using GitHub Actions.
- Monitor infrastructure health using AWS CloudWatch.
- Simulate real-world infrastructure failures.
- Strengthen cloud reliability through resilience testing.
- Develop practical skills in AWS and DevOps practices.

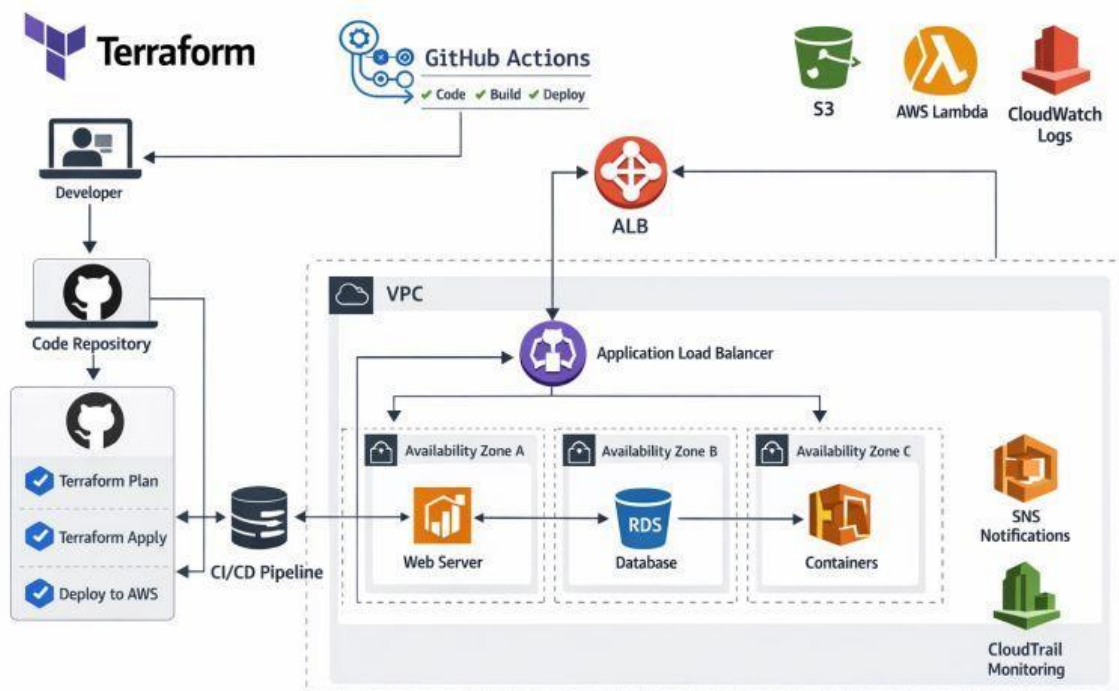
Technology Stack

Category	Technologies
Cloud Platform	AWS EC2, CloudWatch, AWS Lambda, Amazon SQS
Infrastructure as Code	Terraform
Containerization	Docker, Docker Compose
Web Server	NGINX
Application	Flask
CI/CD	GitHub Actions
Operating System	Linux (Ubuntu)
Scripting	Bash

System Architecture

The solution follows a cloud-native DevOps architecture where infrastructure is provisioned automatically and monitored continuously.

Architecture Flow



1. Terraform provisions AWS EC2 infrastructure.
2. Docker Compose deploys multiple application containers.
3. NGINX functions as a reverse proxy.
4. Flask serves the application workload.
5. CloudWatch collects infrastructure and application metrics.
6. AWS Lambda and Amazon SQS generate burst workloads for testing.
7. GitHub Actions automates Docker image builds and deployment workflows.

Infrastructure Provisioning

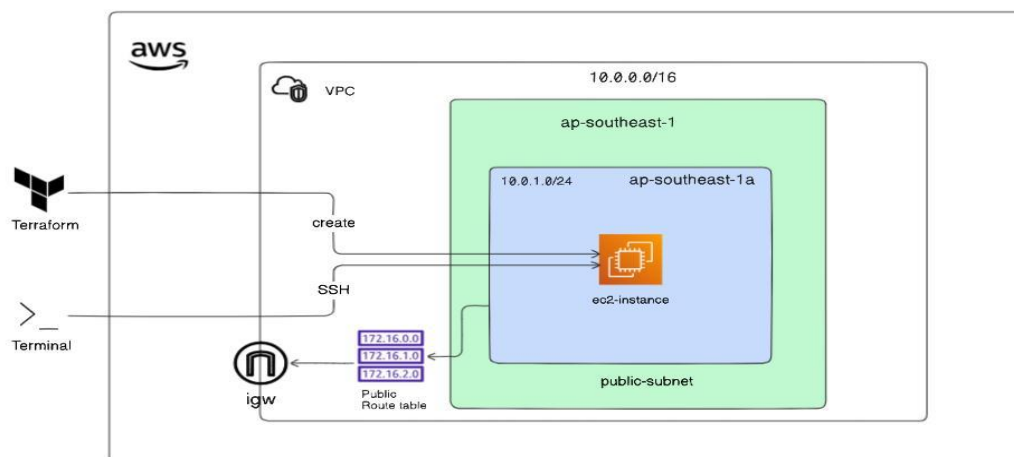
Terraform is used as the Infrastructure as Code solution to provision cloud resources.

Infrastructure Components

- AWS EC2 instances
- Security Groups
- Networking configuration
- Cloud resource automation
- Repeatable infrastructure deployment

Benefits

- Consistent deployments
- Version-controlled infrastructure
- Reduced manual configuration
- Faster environment provisioning



```

61
62 # Create Security Group to allow port 22,80,443
63
64 1 reference
65 resource "aws_security_group" "allow_web" {
66     name           = "allow_web_traffic"
67     description    = "Allow web inbound traffic"
68     vpc_id         = aws_vpc.prod-vpc.id
69
70     ingress = [
71         {
72             description    = "HTTPS"
73             from_port      = 443
74             to_port       = 443
75             protocol       = "tcp"
76             cidr_blocks    = ["0.0.0.0/0"]
77         }
78     ]
79
80     egress = [
81         {
82             from_port      = 0
83             to_port       = 0
84             protocol       = "-1"
85             cidr_blocks    = ["0.0.0.0/0"]
86             ipv6_cidr_blocks = [ ":::/0" ]
87         }
88     ]
89 }

```

```

resource "aws_security_group" "sg_22" {
    name = "sg_22"
    vpc_id = aws_vpc.vpc.id

    ingress {
        from_port = 22
        to_port   = 22
        protocol  = "tcp"
        cidr_blocks = ["0.0.0.0/0"]
    }
}

resource "aws_key_pair" "ssh_key" {
    key_name = "ssh_key"
    public_key = file("ssh_key.pub")
}

```

Containerized Application Deployment

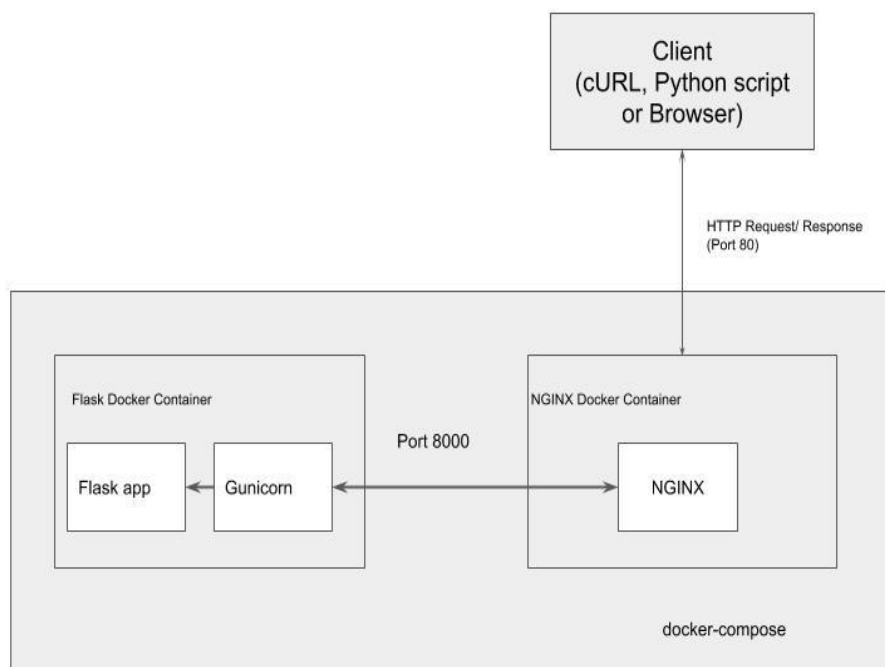
Applications are deployed using Docker and Docker Compose to ensure portability and consistency across environments.

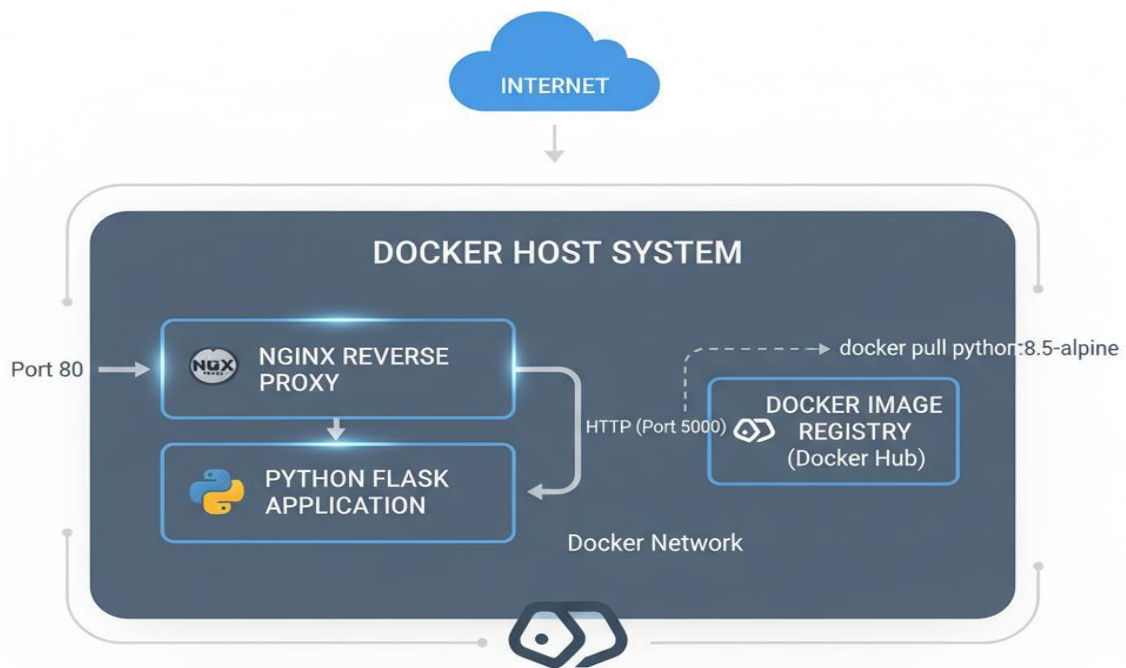
Components

- Flask Application
- NGINX Reverse Proxy
- Docker Containers
- Docker Compose Orchestration

Advantages

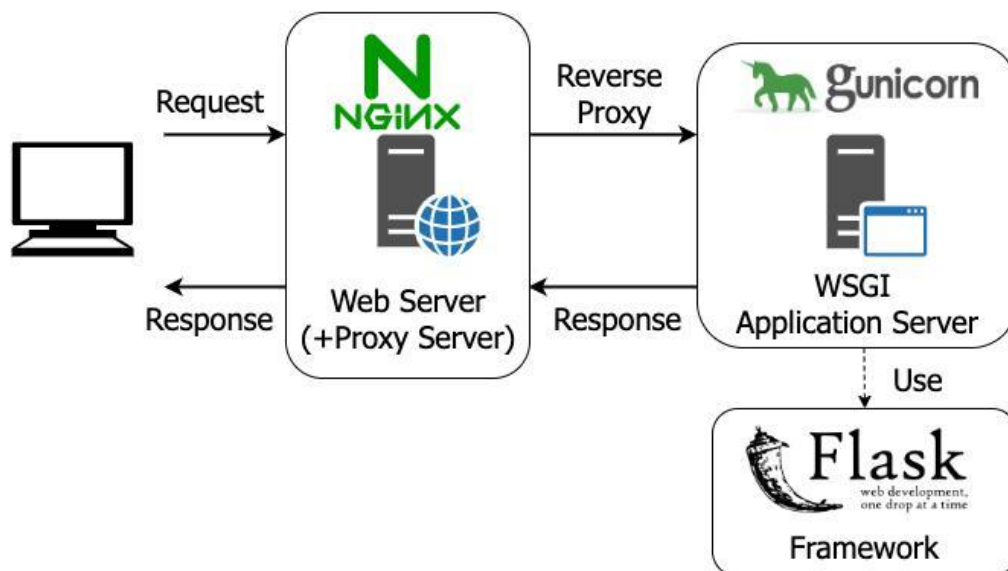
- Isolated application environments
- Simplified deployment process
- Easy scalability
- Consistent runtime behavior





Key Components

- Docker: Containerization Platform
- Python Flask App: Sample Web Application
- Nginx: Reverse Proxy
- DockerFlask: Sample Definition
- DockerFile: Image Definition
- Docker Compose (Optional): Multi-Container Orchestration



Monitoring Implementation

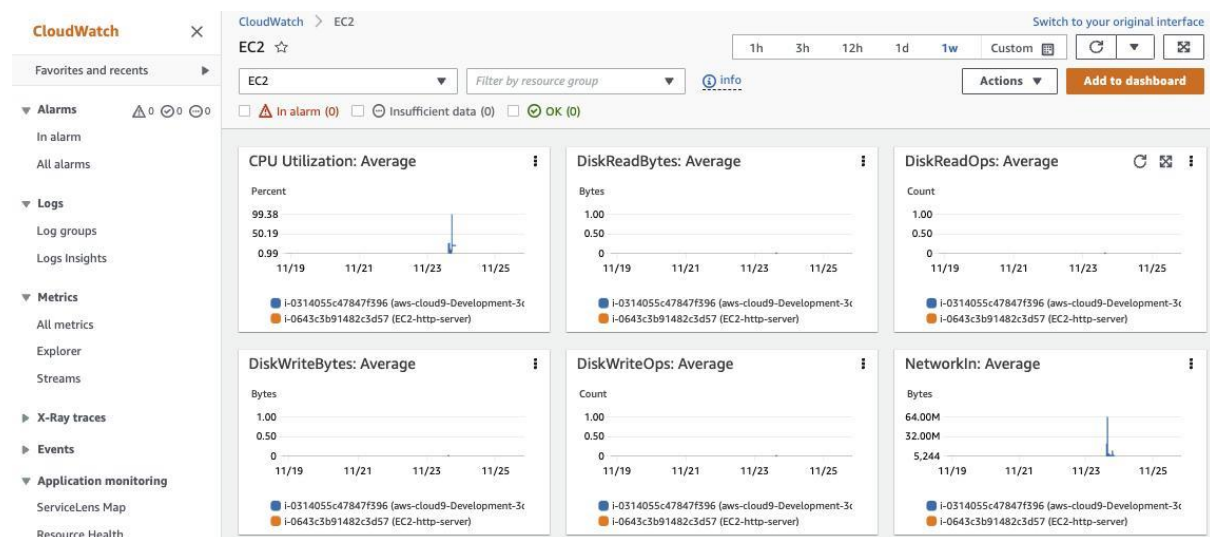
AWS CloudWatch provides continuous monitoring of infrastructure performance and system health.

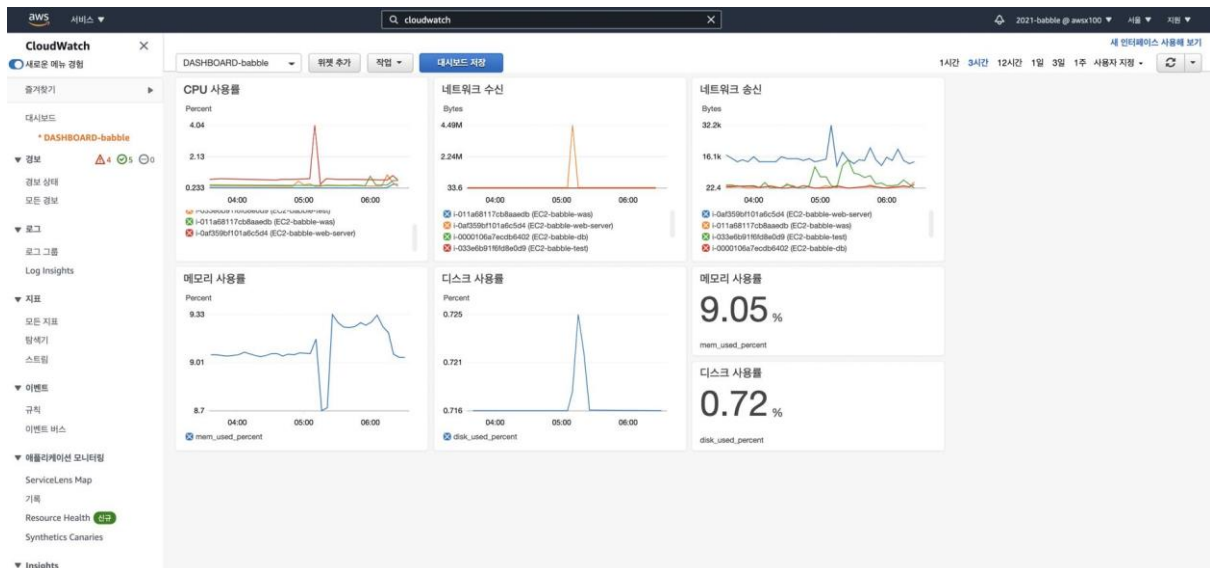
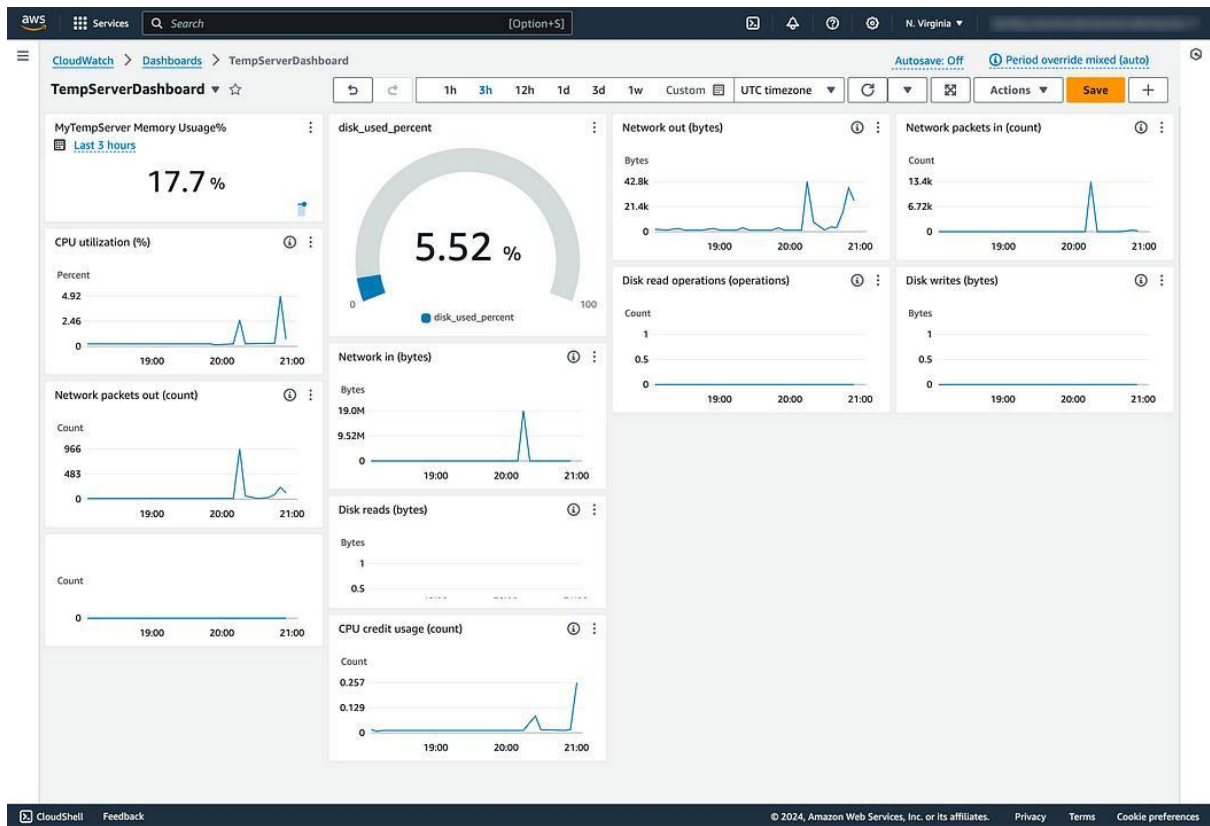
Monitored Metrics

- CPU Utilization
- Memory Usage
- Disk Usage
- Inode Consumption
- System Logs
- Application Logs

Monitoring Benefits

- Real-time infrastructure visibility
- Faster issue detection
- Performance analysis
- Operational troubleshooting





Chaos Engineering Implementation

Chaos engineering techniques are implemented to evaluate infrastructure resilience under controlled failure conditions.

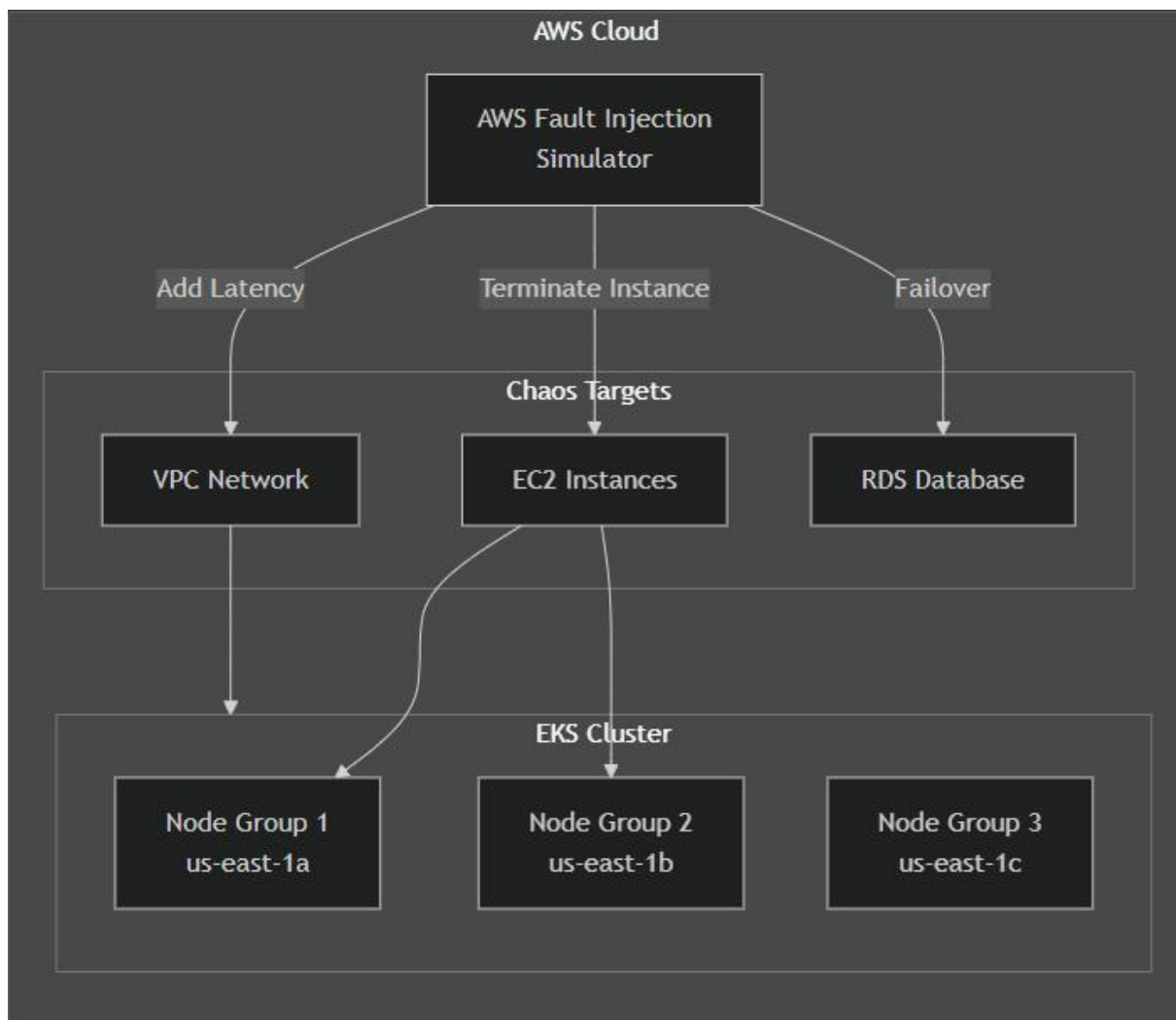
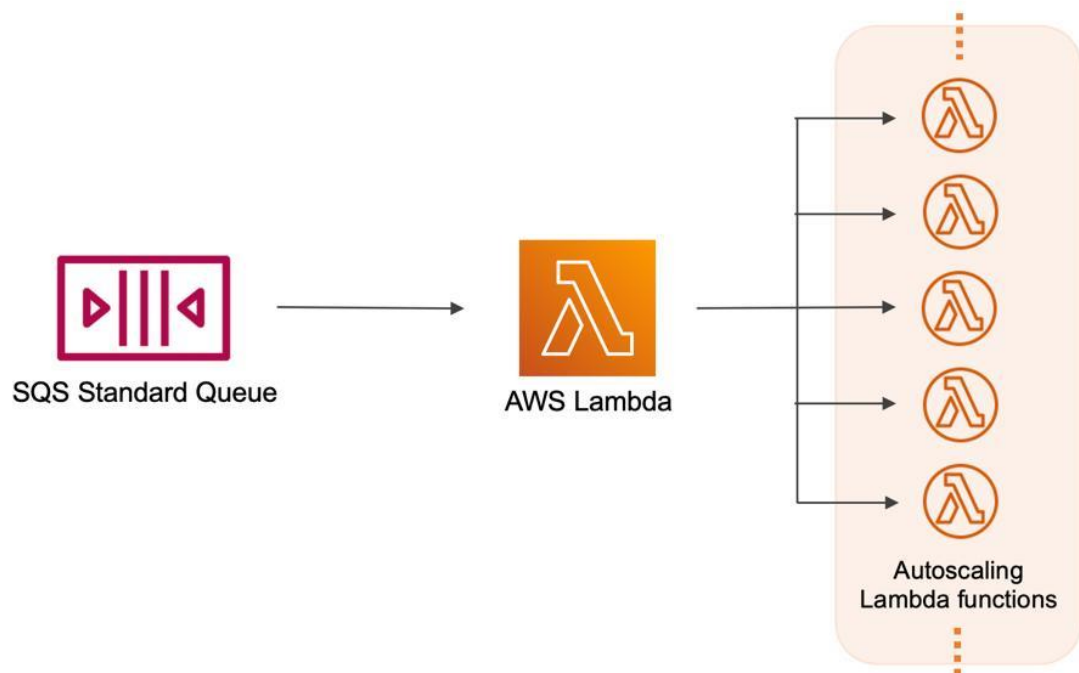
Failure Scenarios

- CPU spike simulation
- Memory exhaustion
- Disk space consumption
- Inode exhaustion
- Burst workload testing using Lambda and SQS

Purpose

These experiments help observe system behavior, identify performance bottlenecks, validate monitoring alerts, and improve infrastructure reliability.





CI/CD Automation

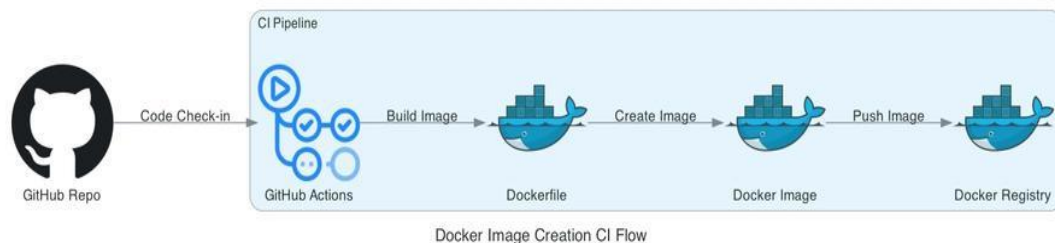
GitHub Actions automates the application delivery pipeline.

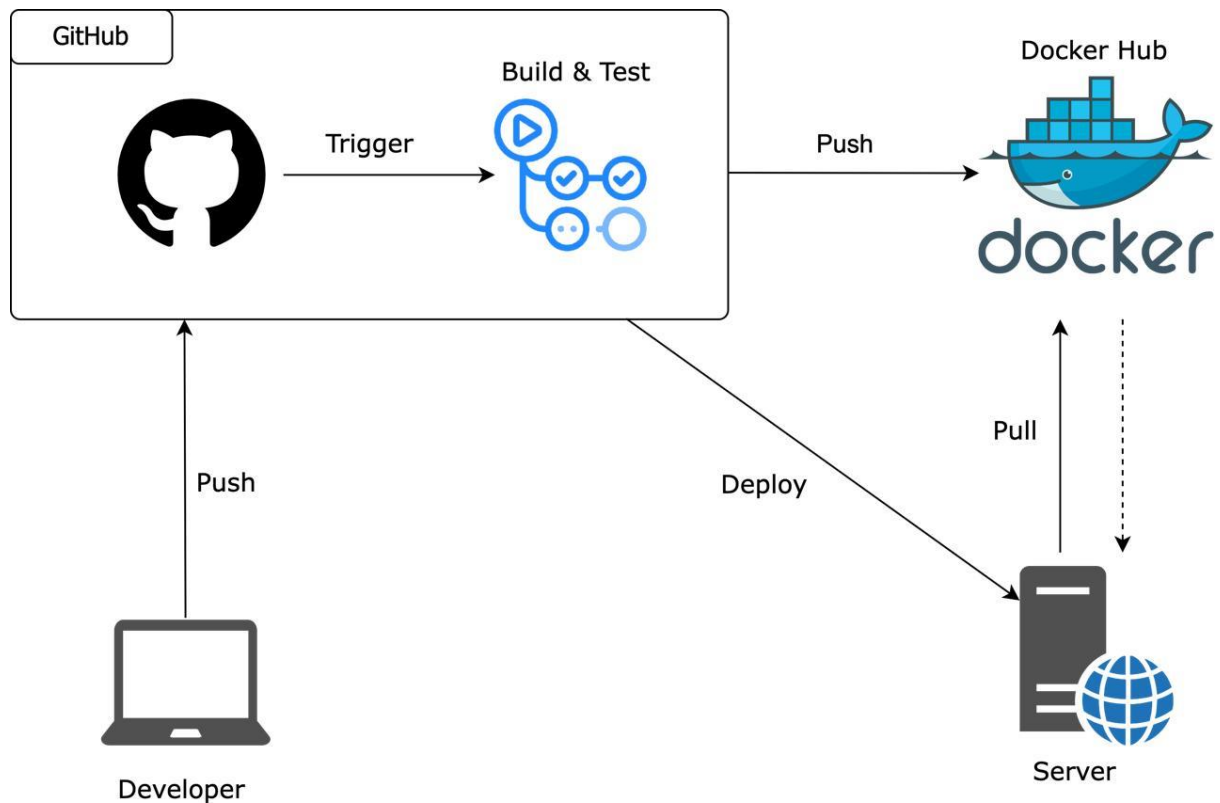
Automated Workflow

- Source code validation
- Docker image build
- Versioned image publishing
- Deployment automation
- Rollback support through image versioning

Benefits

- Faster software delivery
- Reduced manual effort
- Improved deployment consistency
- Better release management





Project Workflow

The implementation follows a structured deployment process.

1. Clone the repository.
2. Configure AWS credentials.
3. Initialize Terraform.
4. Provision infrastructure using terraform apply.
5. Deploy containers with Docker Compose.

6. Configure monitoring.
7. Execute chaos testing scenarios.
8. Analyze CloudWatch metrics and logs.

Key Features

- Infrastructure provisioning with Terraform.
- AWS EC2 cloud deployment.
- Dockerized application deployment.
- NGINX reverse proxy configuration.
- CloudWatch monitoring integration.
- Lambda and SQS burst workload testing.
- GitHub Actions CI/CD automation.
- Chaos engineering implementation.
- Infrastructure troubleshooting and observability.

Challenges and Solutions

Challenge	Solution
Infrastructure provisioning	Automated using Terraform
Application deployment	Standardized with Docker Compose
Performance monitoring	Integrated AWS CloudWatch
Stress testing	Implemented controlled failure simulations
Deployment consistency	Automated using GitHub Actions

Skills Demonstrated

This project demonstrates practical knowledge of:

- AWS Cloud Infrastructure
- Terraform (Infrastructure as Code)
- Docker
- Docker Compose

- NGINX
- Flask Deployment
- AWS CloudWatch
- GitHub Actions
- Linux Administration
- Bash Scripting
- Cloud Monitoring
- Troubleshooting
- DevOps Automation
- Chaos Engineering Fundamentals

Project Outcomes

The project successfully demonstrates how modern cloud infrastructure can be automated, monitored, and tested using DevOps best practices. It highlights practical experience with cloud deployment, infrastructure automation, container orchestration, monitoring, and resilience testing, making it highly relevant for Cloud Engineer, DevOps Engineer, AWS Engineer, and Site Reliability Engineer (SRE) roles.

Repository

GitHub Repository

`git@github.com:ghanushv4/chaos-lab.git`

This project reflects hands-on implementation of cloud infrastructure automation, monitoring, CI/CD, and chaos engineering practices aligned with industry-standard DevOps workflows.